Fabric execution in Dev apps: MCP vs CLI

1 Intro

When using "Dev apps" like Claude Code, it can be confusing to understand how the workflow is actually executed when working with Fabric (or other services).

For example:

- Is the request executed locally?
- Does it go through MCP (Model Context Protocol)?
- Or does the application run a CLI (Command Line Interface) command?

Understanding this execution flow helps clarify what the "**Model**" does vs what the "**Dev app**" does.

2 WHAT IS AN AI DEV APPLICATION?

A "Dev app" is a program that lets you interact with an AI model and use it to perform development tasks. It acts as the interface between the user, the AI model, and the tools available on your system.

Examples include:

- Claude Desktop
- Claude CLI
- Claude Extension in VS Code
- Cursor
- ...

These applications can:

- send requests to the AI model
- receive model outputs (including execution plans)
- execute actions using available tools

Terminology (3 words):

- **Request:** what the user asks for.
- **Execution plan:** what the model prepares.
- **Action:** what the Dev app executes.

Important point:

- In tool-enabled flows, the model (Opus, ChatGPT, etc.) primarily proposes the execution plan.
- The "Dev app" executes the action using available tools.

3 HOW THE MODEL AND THE AI DEV APPLICATION WORK TOGETHER

When you ask something to a "Dev app", three components are involved:

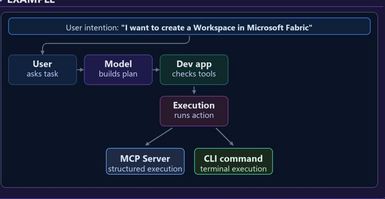
- User – describes the task, for example "create a workspace in Fabric"
- Model – constructs the execution plan describing what should be done
- "Dev app" – decides how to execute the action

To execute actions there are 2 ways:

- MCP
- CLI

In this setup, the "Dev app" selects the execution path **(MCP** or **CLI)** based on available tools and executes the action.

4 EXAMPLE



5 STEP 1 - MODEL OUTPUT (PLANNING)

When tools are exposed by the Dev app, the model can propose an execution plan using those tools.

- If MCP is available, model execution plan will be:
  fabric_create_workspace(name="test workspace")
- If MCP is unavailable and CLI is installed, command from model execution plan will be:
  fab create testWorkspace.Workspace

Note: Typical behavior if no MCP and no Fabric CLI:

- Model can still propose a plan/instructions
- Dev app cannot run the action automatically
- It should return a "missing tools" response + setup guidance

6 STEP 2 - COMMAND EXECUTION

Depending on the planning output, the "**Dev app**" chooses the right tool and uses one of the following execution options.

Example MCP request:

MCP call sent by "Dev app" (JSON-RPC 2.0)

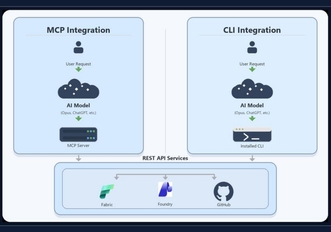
```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "fabric_create_workspace",
    "arguments": { "name": "test workspace" }
  }
}
```

Example CLI command:

Terminal

```
PS C:\>  fab create testWorkspace.Workspace
```

7 VISUAL EXPLANATION



8 MCP (MODEL CONTEXT PROTOCOL)

With MCP, the "Dev app" sends the execution plan as a structured request to a Fabric MCP server.

The MCP server then executes the action in Fabric, and acts as a controlled interface between the model and Fabric.

Advantages:

- structured interaction with Fabric
- controlled execution
- easier permission and integration management

This approach is structured and controlled but requires running and configuring MCP servers.

9 CLI (COMMAND LINE INTERFACE)

With CLI, the "Dev app" relies on installed command line tools.

The model generates a command, and the "Dev app" executes it in the terminal.

Advantages:

- no MCP server selection or configuration required
- uses existing CLI tools already installed

This approach is simple and flexible but depends on the required CLIs being installed.

10 PRACTICAL DIFFERENCE

MCP is often used for structured integrations or production environments where tools are exposed through controlled interfaces.

CLI is often used during development because it is simple and flexible.

When both are available, "Dev apps" may prefer MCP and fall back to CLI if needed.

11 CONCLUSION

MCP and CLI are two execution paths, but both typically end up calling Fabric REST APIs.

- MCP servers typically call Fabric REST APIs
- Fabric CLI commands interact with Fabric service endpoints

In both cases, the Fabric REST API is the underlying interface used to interact with the service.

The difference between MCP and CLI is therefore how the execution plan is executed, not how Fabric itself is accessed.